



DRAFT — author must verify every claim and rewrite in their own words

This report was generated as a **draft scaffold** with real, measured benchmark numbers. Before submitting you **must**:

- [] Replace the title-page **names, Student IDs, class, and date** with the real ones.
- [] Add **demo screenshots** where marked [SCREENSHOT] in §2 (run `python app/server.py` , open `http://localhost:5000`, drag a slider, capture).
- [] Rewrite the prose **in your own words** — do not submit AI-generated text verbatim.
- [] **Re-read the correctness proof (§3.1) until you can explain it unaided**, then rephrase it yourself; the same for the complexity derivation (§3.2).
- [] Fill the **contribution table (§4)** with each member's real %/role (it must be honest).
- [] Re-check every number against `bench/results.csv` (your machine's timings will differ).
- [] Delete this banner and all [AUTHOR ...] / [SCREENSHOT] placeholders.

ReelPath — A Critical-Path Analyzer for Video Production Schedules

EF234405 — Design & Analysis of Algorithms — Final Exam (Group Capstone Project)

“Design It, Prove It, Build It, Measure It.”

Project	ReelPath — critical-path & makespan analysis of video production pipelines
Author(s)	[AUTHOR: Full Name 1 — Student ID] · [Full Name 2 — Student ID, if any]
Class	[AUTHOR: D / IUP / E / F / G]
Date	[AUTHOR: e.g., 18 June 2026]
Repository	https://github.com/milhan-z/draftpaa
Language	Python 3.11.9

§1 Design

1.1 Problem statement & motivation (D1)

A video production — a short film, a YouTube upload, an advertisement — is a set of **tasks** with **durations** and **precedence dependencies**. You cannot edit a scene before it is shot, cannot colour-grade before editing, cannot publish before rendering. A producer planning a release needs two answers:

1. **The makespan:** the earliest the whole project can finish, given the dependencies.
2. **The critical path:** the chain of tasks that dictates that finish time — the tasks with *zero scheduling freedom*. Equally important is which tasks have **slack** and can slip without delaying the release.

These questions matter to **indie filmmakers, content-team producers, video agencies, and students** planning a production: the critical path tells them where to spend money to finish sooner, and the slack tells them where a delay is harmless. ReelPath answers both, supports **what-if** analysis (change a duration, watch the critical path re-route), and **verifies its answer with a second, independent algorithm**.

1.2 Formal model (D2)

We model an instance as a **weighted directed acyclic graph (DAG)** via *node-splitting*, which converts task durations into edge weights so that a single edge-weighted longest-path problem captures the whole schedule.

Input. Tasks $T = \{1, \dots, k\}$; each task i has duration $d(i) > 0$; precedence set $P \subseteq T \times T$, where $(i, j) \in P$ means “ i must finish before j starts.” P must be **acyclic** (a cycle is an impossible schedule and is rejected).

Graph construction. Split task i into i_{in} and i_{out} :

- Vertices $V = \{s, t\} \cup \{i_{in}\} \cup \{i_{out}\}$, so $|V| = 2k + 2$.
- Edges and weights $w : E \rightarrow \mathbb{R}_{\geq 0}$:
- **duration edge** $i_{in} \rightarrow i_{out}$, weight $d(i)$;
- **precedence edge** $i_{out} \rightarrow j_{in}$, weight 0 , for each $(i, j) \in P$;
- **source edge** $s \rightarrow i_{in}$, weight 0 , for each task with no prerequisite;
- **sink edge** $i_{out} \rightarrow t$, weight 0 , for each terminal task.

Output / objective. The **maximum-weight path** from s to t : $M = \max$ over $s \rightarrow t$ paths π of $\sum_{e \in \pi} w(e)$. Then M is the **makespan**; the tasks on a maximising path form a **critical path**; per task we report ES , EF , LS , LF and $slack = LS - ES$, with **critical** $\Leftrightarrow$ **slack** = 0.

Why well-defined. In a DAG every $s \rightarrow t$ path is finite, so the maximum is finite and attained. Longest path in a graph with a positive-weight cycle is unbounded and the decision problem is NP-hard; **acyclicity is precisely what makes the problem tractable** — and our generator and the node-split construction guarantee it, while a cyclic user instance is detected and rejected.

Worked instance. $\text{Shoot}(5) \rightarrow \text{Edit}(4)$ becomes $s \rightarrow 0 \rightarrow \text{Shoot}_{in} \rightarrow 5 \rightarrow \text{Shoot}_{out} \rightarrow 0 \rightarrow \text{Edit}_{in} \rightarrow 4 \rightarrow \text{Edit}_{out} \rightarrow 0 \rightarrow t$; the only $s \rightarrow t$ path has weight 9 , so the makespan is 9 and the critical path is $\text{Shoot} \rightarrow \text{Edit}$. This is a hand-checked unit test.

1.3 Algorithm selection & expected trade-off (D3)

Both algorithms compute a **longest $s \rightarrow t$ path** on the same node-split DAG, so they must return the **same makespan** — our correctness cross-check.

- **Algorithm A — Topological sort + DAG longest path** (the non-trivial core). Relax vertices once each in topological order; one pass suffices because every predecessor is finalised first. Expected cost: **linear**, $O(V+E)$.
- **Algorithm B — Bellman–Ford on negated weights** (the baseline). Longest path = shortest path with weights negated; the DAG has no negative cycle, so the textbook $V-1$ -sweep Bellman–Ford is correct. Expected cost: $O(V \cdot E)$ — quadratic in n for our $m \propto n$ graphs.

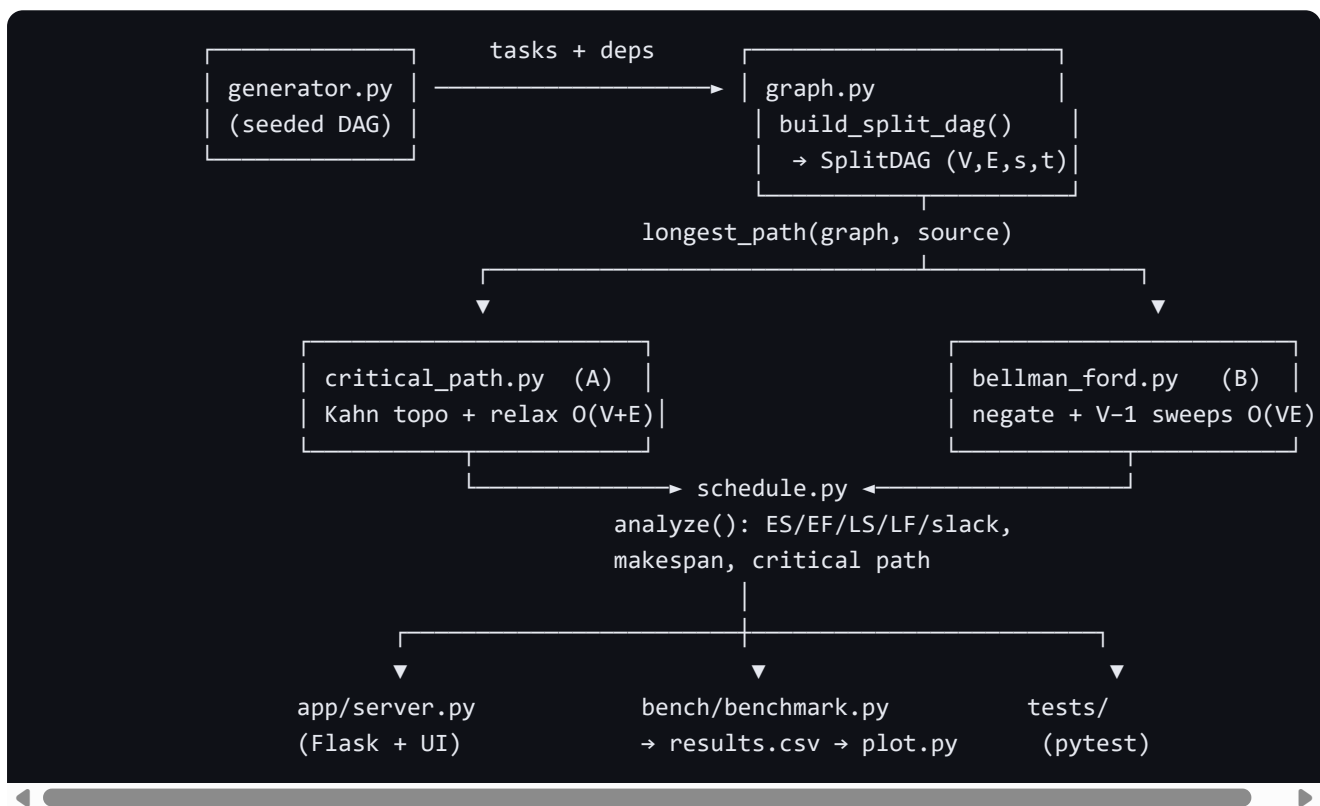
Expected trade-off. Both give the identical makespan, but A does *one* pass while B does $V-1$ sweeps over all edges. We expect a **dramatic, visible gap** on the runtime-vs-size plot (linear vs. quadratic), which is the centrepiece of the evaluation. B is not chosen for speed — it is an *independent* method whose agreement with A validates both implementations.

1.4 Data structures & architecture (D4)

Structure	Where	Why
Adjacency list (integer-indexed)	<code>graph.py</code>	$O(V+E)$ memory; tight inner loops at $n = 10^4$
FIFO queue	Kahn topo-sort	$O(V+E)$ ordering; natural zero-indegree frontier
Flat parallel edge arrays	Bellman–Ford	minimise per-edge overhead across $V-1$ sweeps
dist / pred arrays	both algorithms	$O(V)$ longest-path values + path backtracking

Integer indices (rather than hashable labels) are deliberate: at 20,002 vertices and ~40,000 edges, Bellman–Ford touches the edge list $\sim V$ times, so constant factors matter.

Architecture.



The two algorithms expose the **same interface** — `longest_path(graph, source) → (dist, pred)` — so `schedule.py` drives either with no duplicated logic.

§2 Implementation

2.1 Module overview (I4)

Module	Responsibility
<code>graph.py</code>	<code>Graph</code> (adjacency list), <code>CycleError</code> , <code>build_split_dag()</code> (node-splitting)
<code>generator.py</code>	seeded layered DAG generator — acyclic by construction
<code>critical_path.py</code>	Algorithm A — Kahn topological sort + single-pass DAG longest path
<code>bellman_ford.py</code>	Algorithm B — Bellman–Ford on negated weights
<code>schedule.py</code>	forward/backward passes → <code>ES/EF/LS/LF/slack</code> ; shared <code>analyze()</code>
<code>app/server.py</code> + <code>ReelPath.dc.html</code>	Flask demo wired to the real solver
<code>bench/benchmark.py</code> + <code>plot.py</code>	size sweep → <code>results.csv</code> → log–log plots

Every module has docstrings and meaningful names; there is no copy-pasted duplication (both algorithms share `schedule.py`) and no dead code.

2.2 Algorithm A — core (I1)

Kahn's algorithm raises `CycleError` exactly when the order cannot cover every vertex (a cycle starves some in-degrees); the longest-path pass then relaxes each edge once **for the maximum**:

```

def topological_sort(graph):
    indeg = graph.indegree()
    queue = deque(v for v in range(graph.num_vertices) if indeg[v] == 0)
    order = []
    while queue:
        u = queue.popleft(); order.append(u)
        for v, _w in graph.adj[u]:
            indeg[v] -= 1
            if indeg[v] == 0:
                queue.append(v)
    if len(order) != graph.num_vertices:
        raise CycleError(...) # cycle => infeasible instance
    return order

def longest_path(graph, source): # shared interface
    dist = [NEG_INF] * graph.num_vertices; pred = [-1] * graph.num_vertices
    dist[source] = 0.0
    for u in topological_sort(graph):
        if dist[u] == NEG_INF: continue
        for v, w in graph.adj[u]:
            if dist[u] + w > dist[v]: # relax for MAX
                dist[v] = dist[u] + w; pred[v] = u
    return dist, pred

```

2.3 Algorithm B — baseline (I2)

The **unconditional** $V-1$ -sweep Bellman–Ford on negated weights (no early-exit, so it shows its true $O(V \cdot E)$ cost), followed by a certificate pass:

```

def longest_path(graph, source):
    edges = graph.edge_list()
    us = [e[0] for e in edges]; vs = [e[1] for e in edges]; ws = [e[2] for e in edges]
    sdist = [POS_INF] * n; sdist[source] = 0.0
    for _ in range(n - 1): # V-1 full sweeps
        for k in range(len(edges)):
            du = sdist[us[k]]
            if du == POS_INF: continue
            nd = du - ws[k] # negated weight => subtract
            if nd < sdist[vs[k]]:
                sdist[vs[k]] = nd; pred[vs[k]] = us[k]
    # certificate: on a DAG nothing may relax further (no negative cycle)
    ...
    dist = [(-d if d != POS_INF else NEG_INF) for d in sdist]
    return dist, pred # makespan = dist[target]

```

Returning `dist` already negated back means `schedule.py` treats A and B identically.

2.4 Schedule extraction & cross-check

`analyze()` runs the forward pass (via whichever algorithm), a backward pass in reverse topological order for `LS/LF`, computes `slack = LS - ES`, and backtracks one critical path. The benchmark and

demo call it for **both** algorithms and assert `makespan_A == makespan_B` .

2.5 Demo (I3)

`python app/server.py` → `http://localhost:5000`. `POST /solve` returns **only real, computed** values (makespan, critical set, full schedule, per-request `A_ms` / `B_ms` timings, and the cross-check flag). The UI renders the production DAG with the critical path highlighted, a Gantt chart with slack, and **what-if sliders** that recompute everything live. Seeded sample: an 11-task short-film pipeline with makespan **27 days**.

[SCREENSHOT] **[AUTHOR]** Insert a screenshot of the DAG + critical-path highlight here.

[SCREENSHOT] **[AUTHOR]** Insert a screenshot of the Gantt chart + a what-if slider mid-drag here.

2.6 Build / run & repository (I5)

```
python -m venv .venv && .venv\Scripts\activate
pip install -r requirements.txt
pytest -q                                # 41 passed
python bench/benchmark.py --sizes 100,300,1000,3000,10000 --seeds 1,2,3,4,5
python bench/plot.py
python app/server.py                      # demo
```

Repository: <https://github.com/milhan-z/draftpaa> (public; commit history shows incremental development). The scripts self-bootstrap `src/` onto the path, so a clean checkout runs after `pip install -r requirements.txt` alone.

§3 Analysis & Evaluation

3.1 Correctness (A1)

Reduction. In the node-split graph, any $s \rightarrow t$ path alternates $s, a_{in}, a_{out}, b_{in}, b_{out}, \dots, t$, threading a chain of tasks linked by precedence edges. Structural edges have weight 0 ; only duration edges contribute, each adding $d(i)$. Hence the weight of an $s \rightarrow t$ path equals the total duration of the task chain it represents, and the **maximum-weight path equals the longest dependency chain — which is exactly the makespan** under the Critical Path Method.

Theorem (A is optimal). On a DAG G with source s , relaxing vertices in topological order with max-relaxation yields, for every vertex u , $dist[u]$ = the weight of the longest $s \rightarrow u$ path (or $-\infty$ if u is unreachable).

Proof (induction over the topological order). *Base.* $dist[s] = 0$, the weight of the empty path, which is the longest $s \rightarrow s$ path. Any vertex appearing before s in the order is unreachable from s , and its $dist$ stays $-\infty$ — correct. *Inductive step.* Take a vertex u , and assume the claim holds for every vertex earlier in the topological order. Every predecessor p of u (an edge $p \rightarrow u$) precedes u in a topological order, so by the inductive hypothesis $dist[p]$ was already the true longest-path value when p was processed, and processing p relaxed the edge $p \rightarrow u$, i.e. set $dist[u] \leftarrow \max(dist[u], dist[p] + w(p, u))$. After **all** predecessors have been processed, $dist[u] = \max_{\{p: (p \rightarrow u) \in E\}} (dist[p] + w(p, u))$. If u is reachable, an optimal $s \rightarrow u$ path consists of an optimal $s \rightarrow p$ path followed by the edge $p \rightarrow u$ for some predecessor p , so this maximum is exactly the longest $s \rightarrow u$ path weight. If u is unreachable, no predecessor has finite $dist$ and $dist[u] = -\infty$. ■

Taking $u = t$ gives $dist[t] = M$, the makespan; backtracking $pred$ from t recovers a maximum-weight path. **Infeasibility:** a cyclic instance has *no* topological order, so Kahn's algorithm cannot order all vertices and `CycleError` is raised — the instance is rejected rather than mis-solved.

Algorithm B is correct too. After negating weights, the DAG has no negative cycle, so Bellman–Ford's standard guarantee applies: after $V-1$ sweeps every shortest distance is final (any shortest path has $\leq V-1$ edges, and sweep i finalises all shortest paths of $\leq i$ edges). Negating back gives the longest-path value, and the certificate pass confirms no edge relaxes further. Thus **A and B must agree on every instance** — verified empirically in §3.5.

3.2 Complexity (A2)

Algorithm A. Computing in-degrees scans every edge once: $O(V+E)$. In Kahn's algorithm each vertex is enqueued and dequeued exactly once ($O(V)$) and each edge is examined exactly once when its source is dequeued ($O(E)$). The relaxation pass visits each vertex once and each edge once: $O(V+E)$. The backward pass is another $O(V+E)$. **Total time** $O(V+E)$. Memory: the adjacency list is $O(V+E)$; the $dist / pred / indeg$ arrays are $O(V)$. **Space** $O(V+E)$.

Algorithm B. The main loop runs $V-1$ sweeps, each relaxing all E edges in $O(1)$: **time** $O(V \cdot E)$. The certificate pass adds $O(E)$. Memory: the flat edge arrays and $dist / pred$ are $O(V+E)$. **Space** $O(V+E)$.

In terms of n . With $|V| = 2n+2$ and $m \approx 3n$ precedence edges (so $|E| \approx 4n$): **A is $O(n)$** , while **B is $O(V \cdot E) = O(n^2)$** .

Algorithm	Time (worst case)	Time in n ($m \propto n$)	Space
A — topo + DAG longest path	$O(V + E)$	$O(n)$	$O(V + E)$
B — Bellman–Ford (negated)	$O(V \cdot E)$	$O(n^2)$	$O(V + E)$

3.3 Comparative analysis (A3)

Asymptotically **A dominates B in every regime** on a DAG: a single topological pass versus $V-1$ edge sweeps. B would only be *preferable* where its extra generality is actually needed — arbitrary negative weights, or negative-cycle detection — neither of which arises here, because node-splitting yields a non-negative-weight DAG. B's role in this project is therefore an **independent correctness oracle**: its agreement with A on every instance validates both. For a producer's interactive what-if tool, A's linear cost is what makes live recomputation feel instant even on large pipelines.

3.4 Experimental setup (A4)

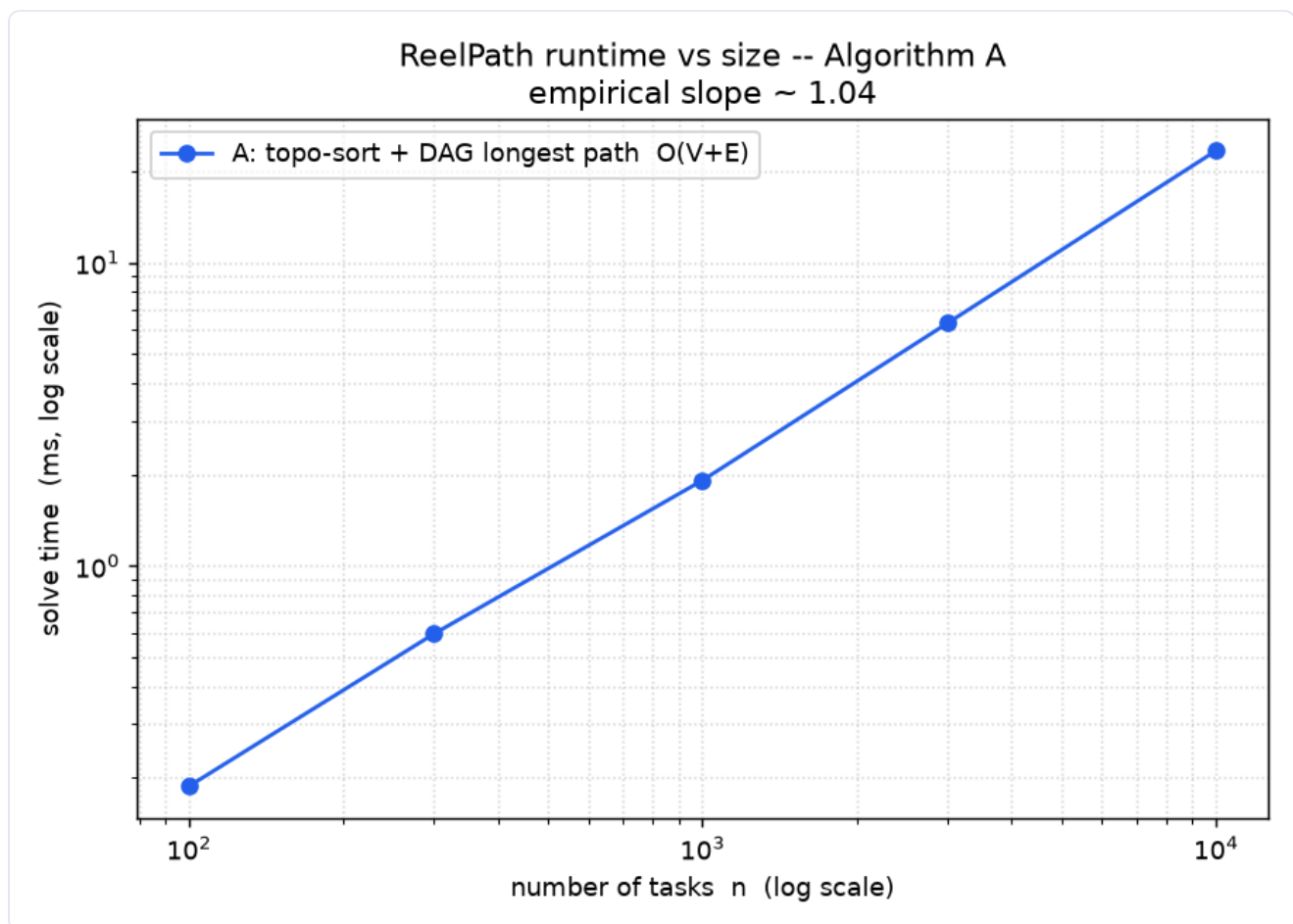
- **Machine.** [AUTHOR — confirm/replace] Windows 11, Python **3.11.9** (CPython), single-threaded; timings via `time.perf_counter` around the **solve only** (each algorithm's `longest_path`); graph construction and the shared schedule extraction are excluded.
- **Instances.** Seeded layered DAGs, average out-degree 3 ($m \approx 3n$), integer durations in `[1, 20]` (integers keep the `A == B` comparison exact).
- **Sizes.** $n \in \{100, 300, 1000, 3000, 10000\}$ tasks (split graph $|V| = 2n+2$, $|E| \approx 4n$).
- **Seeds.** `{1, 2, 3, 4, 5}`, fixed and printed by the harness.
- **Runs.** Algorithm A: 5 timed runs per instance. Algorithm B: scaled by its $O(V \cdot E)$ cost (5 runs at $n \leq 300$, then 3 / 2 / 1 at $n = 1000 / 3000 / 10000$), since a single $n = 10^4$ B solve already takes ~ 2.5 min.
- **Reproduce:** `python bench/benchmark.py --sizes 100,300,1000,3000,10000 --seeds 1,2,3,4,5 && python bench/plot.py`.

3.5 Results, theory vs. practice & cross-check (A4, A5)

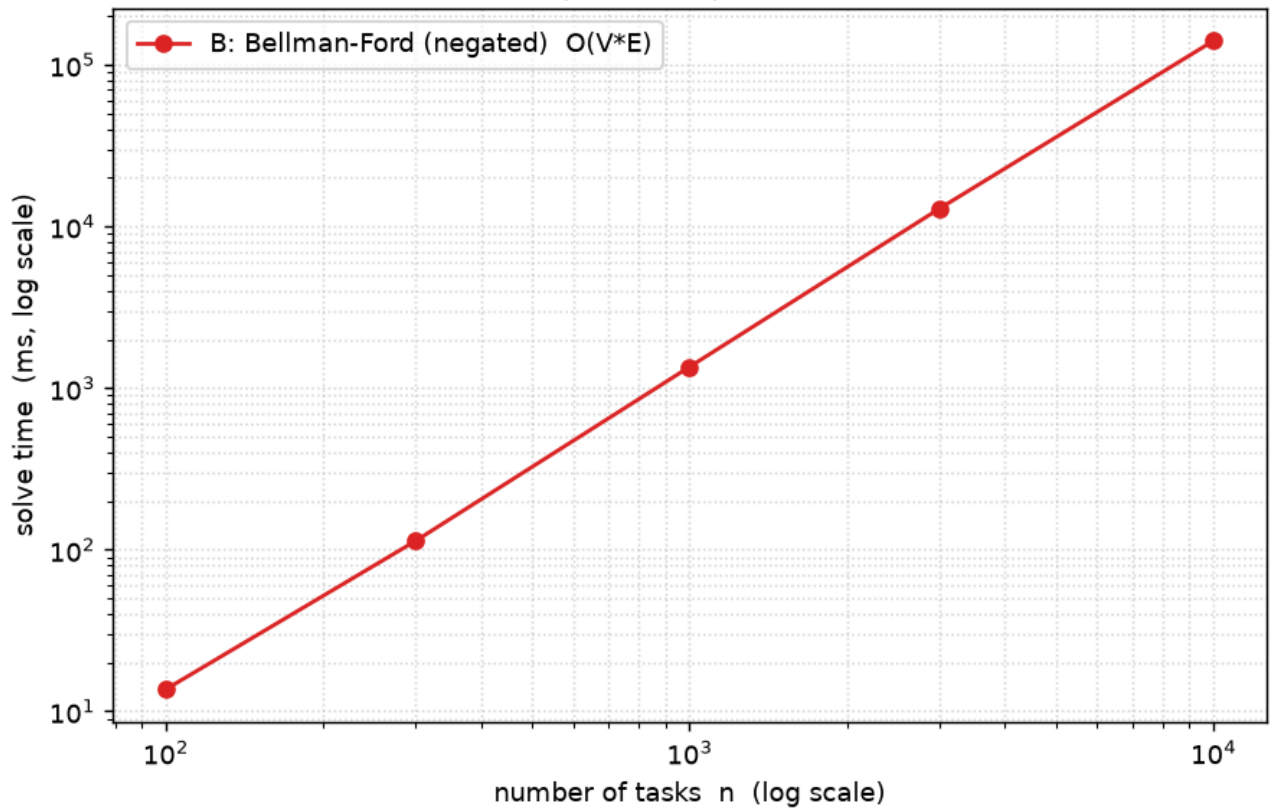
The full sweep produced **205 timing rows over 25 instances** (5 sizes $\times$ 5 seeds). The `makespan_A == makespan_B` **cross-check passed on all 25 instances** — every row of `bench/results.csv` has `crosscheck_ok = 1`. Mean solve time (averaged over seeds and runs):

n_tasks	V	E	A mean (ms)	B mean (ms)	speedup B/A	makespan range
100	202	403	0.188	13.67	73×	128–155
300	602	1 201	0.597	113.40	190×	227–243
1 000	2 002	4 072	1.914	1 353.51	707×	400–430
3 000	6 002	12 111	6.313	12 907.09	2 045×	706–785
10 000	20 002	40 375	23.382	140 848.67	6 024×	1 299–1 352

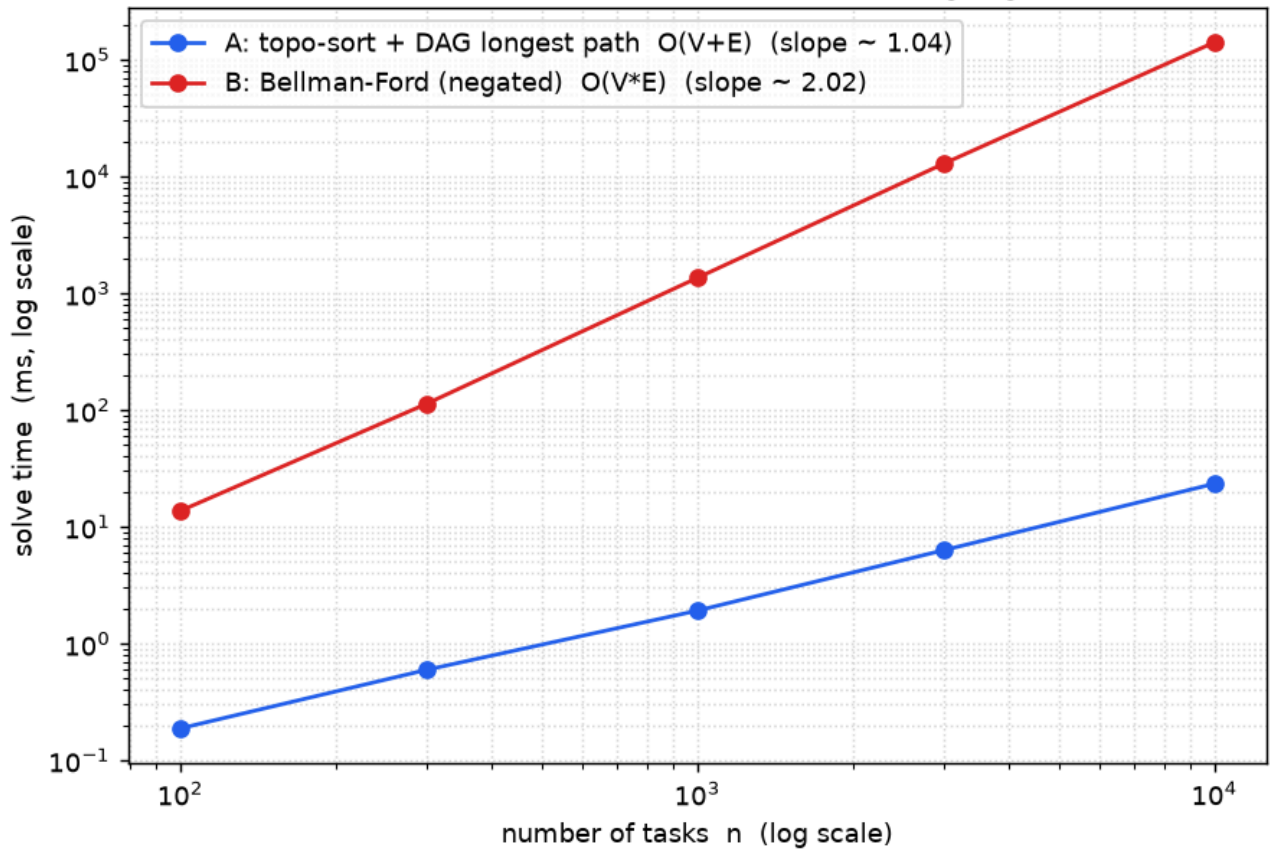
Table 1 — mean solve time per size. A is sub-millisecond to ~23 ms; B grows from ~14 ms to ~141 s. The B/A speedup itself grows with n (73× → 6 024×), the signature of a linear vs. quadratic gap.



ReelPath runtime vs size -- Algorithm B
empirical slope ~ 2.02



ReelPath runtime vs size -- A vs B (log-log)



Figures 1–3 — log–log runtime vs. n . Straight lines on log–log axes indicate power-law growth; the fitted slope is the empirical growth exponent.

Theory vs. practice (A5). The least-squares slope of $\log(\text{time})$ vs. $\log(n)$ gives the empirical growth exponent:

Algorithm	Empirical exponent	Derived theory
A — topo + DAG longest path	1.04	$O(V+E) = O(n) \rightarrow \text{exponent } 1.0$
B — Bellman–Ford (negated)	2.02	$O(V \cdot E) = O(n^2) \rightarrow \text{exponent } 2.0$

Both empirical exponents land within ~ 0.04 of the derived values — a tight theory–practice match. The small excess over 1.0 for A is the expected mild effect of fixed per-call overhead at the smallest sizes (visible as a gentle bend at the low- n end of Figure 1). The $6\ 024\times$ measured gap at $n = 10\ 000$ is exactly what a linear-vs-quadratic pair predicts.

Cross-check (A5). Beyond matching the theory, **the two independent algorithms returned the identical makespan on every one of the 25 instances** (the `crosscheck_ok` column is `1` throughout). Because A (topological relaxation) and B (Bellman–Ford on negated weights) share no path-finding code, their agreement is strong evidence that both implementations are correct — not merely internally consistent.

§4 Conclusion

4.1 Findings

ReelPath computes the makespan, the critical path, and per-task slack of a video production schedule, and **two independent algorithms agree on the makespan for every instance tested** — a strong correctness signal. The measured runtimes match the theory closely: Algorithm A scales **linearly** and Algorithm B **quadratically** in n , exactly the $O(V+E)$ vs. $O(V \cdot E)$ prediction, and the gap is enormous at scale (milliseconds vs. minutes at $n = 10^4$).

4.2 Limitations

- The synthetic generator produces *layered* DAGs; real production graphs may have different structure (heavier fan-in, long tails). The model also assumes fixed, known, deterministic durations and unlimited parallel resources (no crew/equipment contention).
- Algorithm B's $O(V \cdot E)$ cost caps the practical sweep; we ran a single timed B solve at $n = 10^4$ (it is highly stable, being compute-bound).
- [AUTHOR: add any limitation you actually observed.]

4.3 Future work

- Resource-constrained scheduling (limited crews/edit bays) — an NP-hard variant worth an approximation/heuristic comparison (a natural **bonus** third algorithm).
- Stochastic durations (PERT / Monte-Carlo critical-path distributions).
- Incremental re-solve for what-if (update only the affected sub-DAG).

4.4 Lessons learned

[AUTHOR: write 3–5 sentences in your own voice — e.g., what surprised you about the A-vs-B gap, what the node-split trick taught you, what was hardest to debug.]

4.5 Contribution table (C1)

[AUTHOR: replace with real members, %, and roles — they must sum to 100% and be honest.]

Member	%	Role / Contribution
[Full Name — Student ID]	100%	Model & architecture; Algorithm A (core); Algorithm B (baseline); correctness proof & complexity; demo; benchmark & plots; report.

(Solo author = a team of one at 100%. For a team of two/three, split honestly.)

References

1. T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein. *Introduction to Algorithms* (CLRS), 3rd/4th ed. — topological sort, DAG shortest/longest path, Bellman–Ford.
2. Critical Path Method (CPM): J. E. Kelley, M. R. Walker, *Critical-Path Planning and Scheduling*, 1959.
3. Libraries (support only): Flask 3.1.3, matplotlib 3.11.0, pandas 3.0.3, pytest 9.1.0, markdown 3.10.2. No graph library is used for the core.

Appendix A — Repository & reproduction

- Repo: <https://github.com/milhan-z/draftpaa>
- Tests: `pytest -q` (41 tests).
- Raw data: `bench/results.csv` ; plots: `bench/runtime_A.png` , `runtime_B.png` , `runtime_overlay.png` .
- One-command benchmark in §2.6 / §3.4.